Министерство науки и высшего образования РФ

Пензенский государственный университет

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №7

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Обход графа в глубину»

Выполнили студенты группы 25ВВВ2

Лампетов А.С.

Ковтун А.А.

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н., доцент Деев М.В.

Пенза 2026

Тема: Обход графа в глубину

Цель работы

изучение алгоритма обхода графа в глубину и реализация на языке Go его рекурсивного варианта для матрицы смежности, варианта для списков смежности и нерекурсивного варианта на основе стека; оценка сложности обхода.

Лабораторное задание

1. Сгенерировать с помощью генератора случайных чисел матрицу смежности неориентированного графа G и вывести её на экран.
2. Для сгенерированного графа выполнить обход в глубину, реализованный рекурсивно в соответствии с приведённым в методических указаниях описанием.
3.
   – Реализовать процедуру обхода в глубину для графа, представленного списками смежности.
4.
   – Для матричной формы представления графа преобразовать рекурсивную реализацию обхода в нерекурсивную.

Ход работы

Граф хранится в структуре Graph, которая содержит число вершин N и матрицу смежности Mat. Функция NewGraph заполняет верхний треугольник матрицы случайными значениями: ребро между вершинами i и j создаётся с вероятностью p процентов, после чего значение копируется в симметричную ячейку, так что граф получается неориентированным. Вероятность и число вершин (от 1 до 100) задаёт пользователь. Кроме того, каждая диагональная ячейка с той же вероятностью получает единицу, то есть в графе могут появляться петли; на результат обхода они не влияют, так как петля ведёт в

уже посещённую вершину. Вершины нумеруются с единицы, при вводе с клавиатуры номер приводится к индексу массива вычитанием единицы.

```
type Graph struct {
    N   int
    Mat [][]int
}

func NewGraph(n int, p float64) Graph {
    g := Graph{N: n, Mat: make([][]int, n)}
    for i := range g.Mat {
        g.Mat[i] = make([]int, n)
    }
    for i := 0; i < n; i++ {
        for j := i + 1; j < n; j++ {
            if rand.Float64()*100 < p {
                g.Mat[i][j] = 1
                g.Mat[j][i] = 1
            }
        }
    }
    for i := 0; i < n; i++ {
        if rand.Float64()*100 < p {
            g.Mat[i][i] = 1
        }
    }
    return g
}
```

Программа управляется текстовым меню: создание случайного графа, вывод матрицы и списков смежности, три варианта обхода (рекурсивный по матрице, по спискам смежности и нерекурсивный), а также изменение вероятности появления ребра. Все вводимые значения проверяются: при некорректном вводе запрос повторяется. Если граф ещё не создан, обходы не выполняются, и пользователю выводится соответствующее сообщение.

Рекурсивный обход по матрице смежности

Алгоритм реализован в виде двух функций, которые соответствуют двум частям алгоритма из методических указаний. Функция DFSRecursive выполняет подготовительную часть (пп. 1.1–1.3): создаёт массив отметок visited, в котором все вершины сначала не посещены, запускает обход из выбранной пользователем начальной вершины, а затем запускает его из каждой оставшейся непосещённой вершины, чтобы были пройдены все компоненты связности. Функция DFS соответствует процедуре DFS(v) (пп. 2.1–2.8): она помечает вершину как посещённую, записывает её номер в

массив order, затем просматривает v-ю строку матрицы и для каждой смежной непосещённой вершины рекурсивно вызывает себя. Условие v != to исключает петли.

```
func (g Graph) DFS(v int, visited []bool, order []int, index *int) {
    visited[v] = true
    order[*index] = v
    *index++
    for to := 0; to < g.N; to++ {
        if v != to && g.Mat[v][to] == 1 && !visited[to] {
            g.DFS(to, visited, order, index)
        }
    }
}

func (g Graph) DFSRecursive(start int) []int {
    visited := make([]bool, g.N)
    order := make([]int, g.N)
    index := 0
    g.DFS(start, visited, order, &index)
    for v := 0; v < g.N; v++ {
        if !visited[v] {
            g.DFS(v, visited, order, &index)
        }
    }
    return order
}
```

Обход по спискам смежности

Функция AdjacencyList преобразует матрицу в списки смежности: для каждой строки матрицы номера столбцов с единицами добавляются в список соответствующей вершины. Функция DFSAdjacencyList выполняет тот же обход, но вместо просмотра всей строки матрицы перебирает только список соседей текущей вершины. Рекурсивная процедура оформлена как замыкание dfs, которое пользуется общими массивом visited и срезом order.

```
func (g Graph) AdjacencyList() [][]int {
    list := make([][]int, g.N)
    for i := 0; i < g.N; i++ {
        for j := 0; j < g.N; j++ {
            if g.Mat[i][j] == 1 {
                list[i] = append(list[i], j)
            }
        }
    }
    return list
}

func (g Graph) DFSAdjacencyList(start int) []int {
    list := g.AdjacencyList()
    visited := make([]bool, g.N)
    order := make([]int, 0, g.N)
    var dfs func(int)
```

```
    dfs = func(v int) {
        visited[v] = true
        order = append(order, v)
        for _, to := range list[v] {
            if !visited[to] {
                dfs(to)
            }
        }
    }
    dfs(start)
    for v := 0; v < g.N; v++ {
        if !visited[v] {
            dfs(v)
        }
    }
    return order
}
```

Нерекурсивный обход

Чтобы избавиться от рекурсии, стек вызовов заменён явным стеком вершин. Из стека берётся верхняя вершина; если она уже посещена, она пропускается, иначе помечается как посещённая, выводится, а все её непосещённые соседи кладутся в стек. Соседи добавляются в порядке убывания номеров, поэтому из стека они извлекаются в порядке возрастания, и порядок обхода получается таким же, как у рекурсивной версии. Проверка visited при извлечении необходима, потому что одна и та же вершина может быть положена в стек несколько раз раньше, чем будет посещена. Начальная вершина обрабатывается первой, затем проверяются все остальные вершины, чтобы обойти компоненты связности.

```
func (g Graph) DFSIterative(start int) []int {
    visited := make([]bool, g.N)
    order := make([]int, 0, g.N)

    for k := -1; k < g.N; k++ {
        root := k
        if k == -1 {
            root = start
        }
        if visited[root] {
            continue
        }
        stack := []int{root}
        for len(stack) > 0 {
            last := len(stack) - 1
            v := stack[last]
            stack = stack[:last]
            if visited[v] {
                continue
            }
            visited[v] = true
```

```
            order = append(order, v)
            for to := g.N - 1; to >= 0; to-- {
                if g.Mat[v][to] == 1 && !visited[to] {
                    stack = append(stack, to)
                }
            }
        }
    }
    return order
}
```

Оценка сложности

Пусть n — число вершин графа, m — число рёбер. В рекурсивной версии для матрицы смежности каждая вершина посещается один раз, и при посещении просматривается вся её строка из n элементов, поэтому время работы составляет $O(n^2)$ независимо от числа рёбер. При использовании списков смежности при посещении вершины просматривается только список её соседей, а суммарная длина всех списков равна 2m, поэтому сам обход выполняется за O(n + m). Построение списков из матрицы в программе требует просмотра всей матрицы и занимает $O(n^2)$, поэтому для матричного ввода выигрыш получается только при многократных обходах, а для графа, изначально заданного списками, обход выполняется за O(n + m). Нерекурсивная версия по матрице, как и рекурсивная, имеет сложность $O(n^2)$: каждая вершина раскрывается один раз и просматривает строку из n элементов, а число добавлений в стек не превышает суммарной степени вершин. Дополнительная память во всех вариантах составляет O(n) под отметки и порядок обхода; для рекурсивных версий к ней добавляется глубина рекурсии до n в худшем случае (граф-цепочка), для нерекурсивной — размер стека, не превышающий 2m.

Результаты работы программы

Ниже приведены примеры обхода. Первый пример — граф из четырёх вершин с рёбрами 1–2, 1–3 и 2–4, начальная вершина 1.

```
Матрица смежности:
      1  2  3  4
  1   0  1  1  0
  2   1  0  0  1
  3   1  0  0  0
  4   0  1  0  0
```

Рисунок 1 — Сгенерированная матрица смежности

```
Списки смежности:
1: 2, 3
2: 1, 4
3: 1
4: 2
```

Рисунок 2 — Списки смежности графа

Обход начинается с вершины 1. В её строке первой найдена непосещённая вершина 2, из неё осуществляется переход в вершину 4. У вершины 4 единственный сосед 2 уже посещён, и обход возвращается к вершине 1, у которой остаётся непосещённой вершина 3. Три варианта обхода дают один и тот же порядок.

```
Рекурсивный обход в глубину по матрице смежности:
Порядок посещения вершин: 1 -> 2 -> 4 -> 3

Обход в глубину по спискам смежности:
Порядок посещения вершин: 1 -> 2 -> 4 -> 3

Нерекурсивный обход в глубину с помощью стека:
Порядок посещения вершин: 1 -> 2 -> 4 -> 3
```

Рисунок 3 — Результаты обхода графа из четырёх вершин

Второй пример — несвязный граф из шести вершин с рёбрами 1–2, 2–3 и 4–5; вершина 6 изолирована. Начальная вершина 3.

```
Матрица смежности:
      1  2  3  4  5  6
  1   0  1  0  0  0  0
  2   1  0  1  0  0  0
  3   0  1  0  0  0  0
  4   0  0  0  0  1  0
  5   0  0  0  1  0  0
  6   0  0  0  0  0  0
```

Рисунок 4 — Матрица смежности несвязного графа

```
Списки смежности:
1: 2
2: 1, 3
3: 2
4: 5
5: 4
6: нет смежных вершин
```

Рисунок 5 — Списки смежности несвязного графа

Обход из вершины 3 проходит первую компоненту связности (3, 2, 1), после чего алгоритм находит непосещённую вершину 4 и обходит вторую компоненту, а затем отдельно посещает изолированную вершину 6.

```
Рекурсивный обход в глубину по матрице смежности:
Порядок посещения вершин: 3 -> 2 -> 1 -> 4 -> 5 -> 6
```

Рисунок 6 — Обход несвязного графа с начальной вершиной 3

Во всех случаях результаты, выданные программой, совпали с ожидаемыми.

Вывод

В ходе выполнения лабораторной работы был изучен алгоритм обхода графа в глубину, и на языке Go реализованы его рекурсивный вариант для матрицы смежности, вариант для списков смежности и нерекурсивный вариант с использованием стека. Все три реализации дают одинаковый порядок посещения вершин и корректно обходят несвязные графы. Анализ показал, что обход по матрице смежности имеет сложность $O(n^2)$, а по спискам смежности — $O(n + m)$. Результаты работы программы совпали с ожидаемыми, что подтверждает корректность реализации.